Core Functionality & Logic

Requirements: The bot fulfills the proposal's likely goals by offering a functional interface for exploring Astana via 10 detailed categories such as Cafes, Cinema, Karaoke, Oceanarium, Museums, Restaurants, Bowling, Coworkings, Exhibitions, and Parks. All places divided into purpose types, paid places divided into price ranges.

Robustness: By utilizing the aiogram framework and predefined reply keyboards, the bot prevents "invalid input" errors by limiting user choices to valid options.

Logic Optimization: The use of Python dictionaries to store location data ensures efficient data retrieval, which is the optimal choice for this type of application.

Code Quality & Standards

Modularity & Architecture:

The project is built with a focus on separation of concerns. Instead of a similar based script, the logic is divided into modular components. The bot's data (locations, descriptions, and addresses) is isolated from the command handlers, making it easy to update the database without touching the core logic. Each handler is designed as a standalone asynchronous function, which promotes reusability and simplifies debugging.

Readability & Maintainability:

The codebase strictly adheres to PEP 8 standards to ensure professional quality. We prioritized clear, descriptive naming conventions using snake_case for all functions and variables (e.g., process_category_selection), which makes the code self-explanatory. For more complex logic, such as the dynamic keyboard generation, we have included docstrings and inline comments to assist other developers in understanding the data flow.

Performance Optimization:

By leveraging the aiogram library's asynchronous capabilities, the bot can handle multiple user requests concurrently without blocking the execution thread. This ensures a smooth user experience even during peak interaction times.

Version Control & Collaboration

Contribution Balance:

The project successfully maintained a balanced workload by dividing responsibilities based on specialization:

Zhamilya (Lead Developer): Responsible for the core architecture using the aiogram framework, implementing asynchronous handlers, and designing the state management logic.

Danial & Symbat (Data & Content Engineers): Responsible for research, data collection, and structuring the extensive database of Astana's landmarks, as well as designing the user-interface flow (button maps and navigation logic).

Development Process & Integration:

Our team adopted a **"Local-First" development strategy**. To ensure a stable and bug-free repository, the data collection and core coding phases were conducted in parallel local environments:

Research & Prep Phase: Danial and Symbat compiled and formatted all location data into structured Python dictionaries, while Zhamilya developed the bot's structural framework. Symbat completed visual presentation part, Danial made technical documentation.

Integration Day: Once both the data and the engine were ready, the team held an intensive integration session. All localized components were merged, tested for compatibility, and uploaded to the repository.

Commit Quality: Rather than uploading fragmented, non-functional code, we focused on committing "milestone-ready" versions. This approach reflects a professional workflow where the final repository represents a fully tested and synchronized product of collaborative offline preparation.